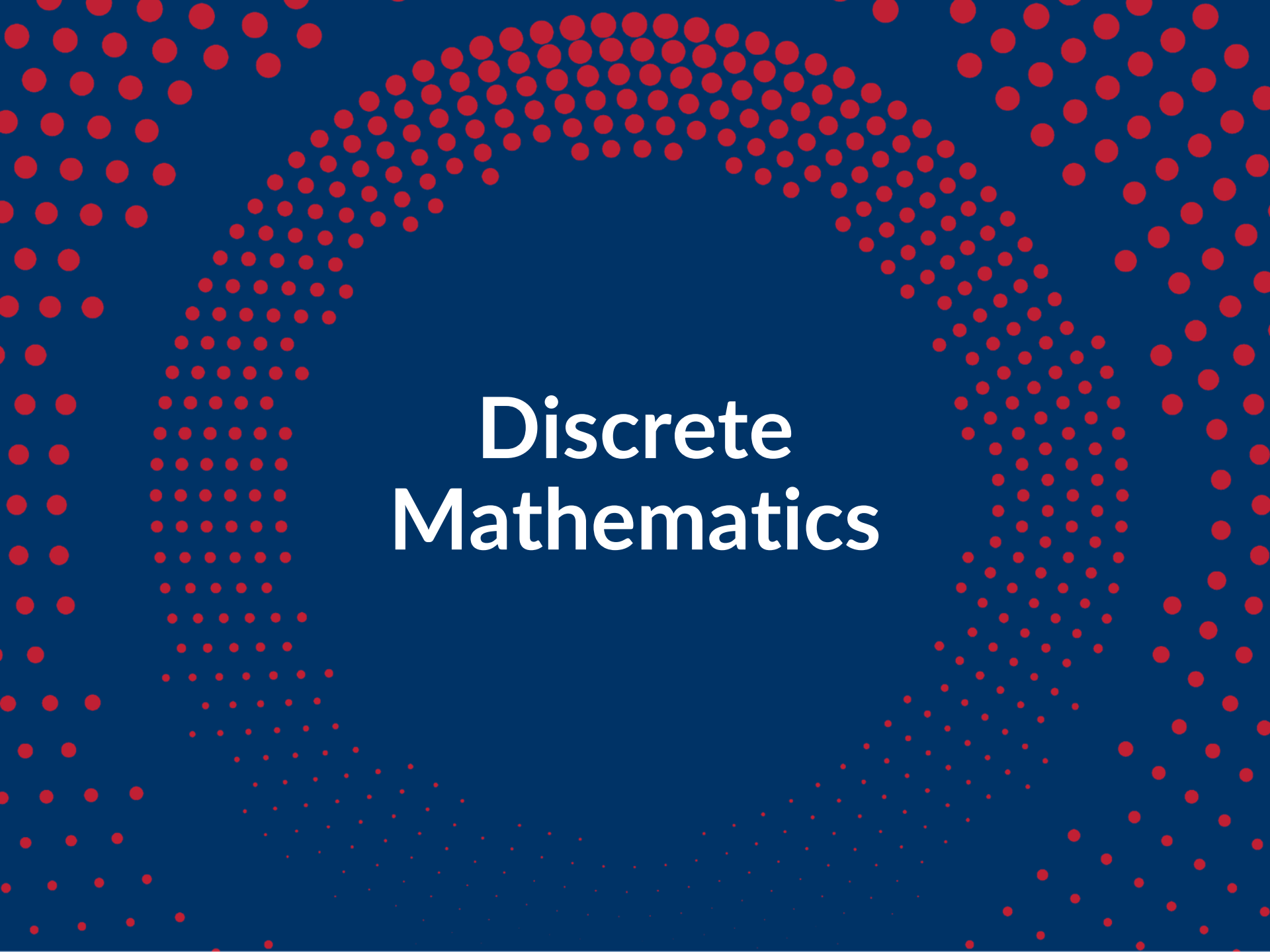


The background of the entire image is a dark blue field filled with a pattern of red dots of varying sizes. These dots are arranged in a way that they form a large, faint, stylized arch or wave shape that spans the width of the image, framing the central text.

HUST

ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

ONE LOVE. ONE FUTURE.



Discrete Mathematics

PART 1

COMBINATORIAL THEORY

(Lý thuyết tổ hợp)

PART 2

GRAPH THEORY

(Lý thuyết đồ thị)

CONTENT

- Chapter 1. Fundamental concepts
- Chapter 2. Graph representation
- Chapter 3. Graph Traversal
- Chapter 4. Tree and Spanning tree
- Chapter 5. Shortest path problem
- **Chapter 6. Maximum flow problem**



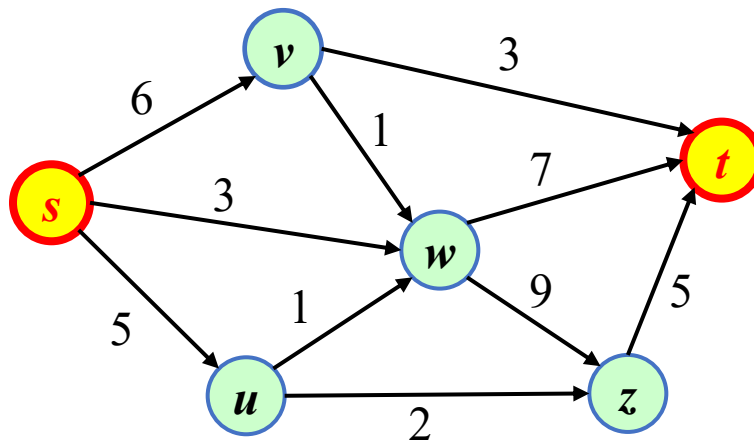
1. Problem description and applications
2. Cut
3. Residual graph and Augmenting path
4. Ford-Fulkerson algorithm
5. Edmond-Karp algorithm
6. Some applications

Network

Network is a directed graph $G = (V, E)$:

- There is only one vertex s without any incoming arcs, called **source vertex** and only vertex t without any outgoing arcs called **target vertex**.
- Each edge e of G is assigned a nonnegative value $c(e)$ which is called capacity of e .

Example:



Flow in network

Definition. Flow f in network $G=(V,E)$ is to assign value $f(e)$ on each edge e ($f(e)$ is flow on edge e) so that following conditions are satisfied:

1) Capacity rule:

For each edge e , $0 \leq f(e) \leq c(e)$

2) Conservation Rule (Điều kiện cân bằng luồng): Each $v \neq s, t$

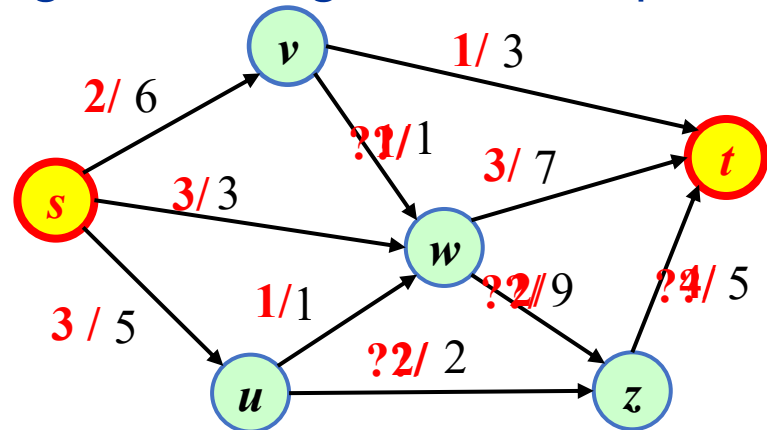
$$\sum_{e \in E^-(v)} f(e) = \sum_{e \in E^+(v)} f(e)$$

$E^-(v)$ and $E^+(v)$ are sets of arcs entering and leaving vertex v , respectively.

Conditions 1) and 2) are satisfied $\Rightarrow$

f is flow in network

e.g. oil flowing through
pipes, internet routing



Flow in network

Definition. Flow f in network $G=(V,E)$ is to assign value $f(e)$ on each edge e ($f(e)$ is flow on edge e) so that following conditions are satisfied:

1) Capacity rule:

For each edge e , $0 \leq f(e) \leq c(e)$

2) Conservation Rule (Điều kiện cân bằng luồng): Each $v \neq s, t$

$$\sum_{e \in E^-(v)} f(e) = \sum_{e \in E^+(v)} f(e)$$

Definition. Value of flow f is

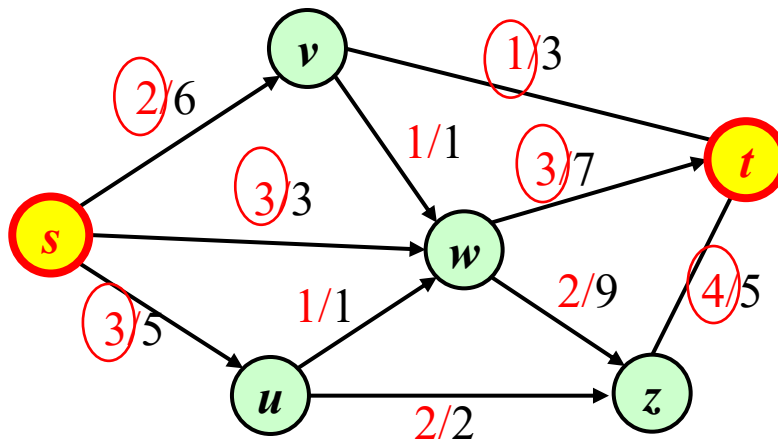
$$val(f) = \sum_{e \in E^+(s)} f(e) \stackrel{(*)}{=} \sum_{e \in E^-(t)} f(e)$$

(Equation $(*)$ is obtained by summing up all the conservation rules.)

Example:

- 2 values on each edge: value of flow on edge is in red, the other is capacity of edge.
- Conditions 1) and 2) are satisfied $\Rightarrow f$ is flow on the network.
- Value of flow:

$$8 = f(s,v) + f(s,u) + f(s,w) = f(v,t) + f(w,t) + f(z,t)$$



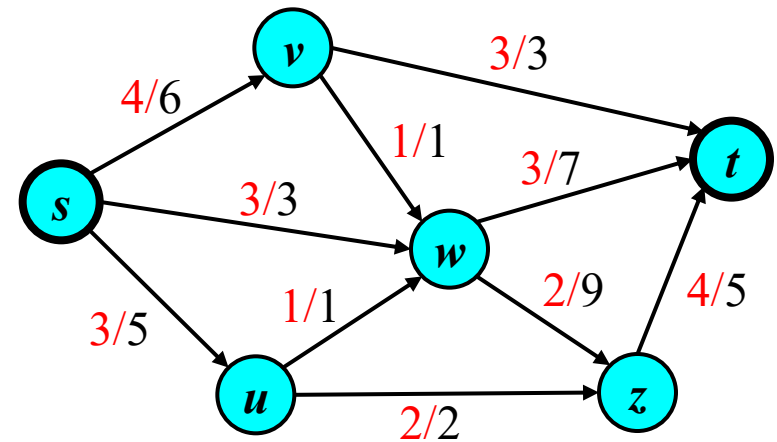
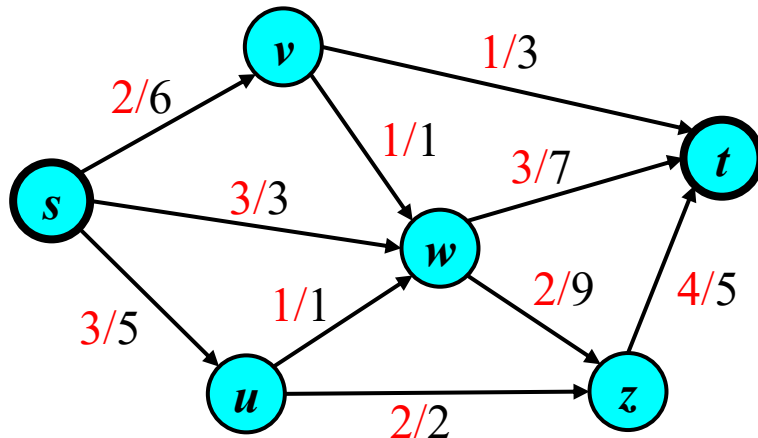
$$val(f) = \sum_{e \in E^+(s)} f(e) \stackrel{(*)}{=} \sum_{e \in E^-(t)} f(e)$$

Maximum Flow problem

A flow in network G is called maximum flow if among all flows in G , it is the one having maximum value.

Maximum flow problem is the problem aiming to determine a feasible flow through the network with maximum value

Flow with value $8 = 2 + 3 + 3 = 1 + 3 + 4$



Maximum flow with value $10 = 4 + 3 + 3 = 3 + 3 + 4$

Some applications

Network	Vertex	Arc	Flow
Network	transaction stations, computers, satellites	cables	voice, video, packets
Electrical network	gates, registers, processors	conductors	Power circuit
Mechanical	joints	rods, beams, springs	heat, energy
Irrigation	pumping stations, water source	pipelines	Liquid, water
Finance	bank	transitions	money
Traffic	airport, station	highways, flights	Goods, customers
Chemistry	sites	bonds	energy



Contents

1. Problem description and applications

2. Cut

3. Residual graph and Augmenting path

4. Ford-Fulkerson algorithm

5. Edmond-Karp algorithm

6. Some applications



Cut

Cut is a partition vertex set of the network into 2 disjoint sets S and T with $s \in S, t \in T$.

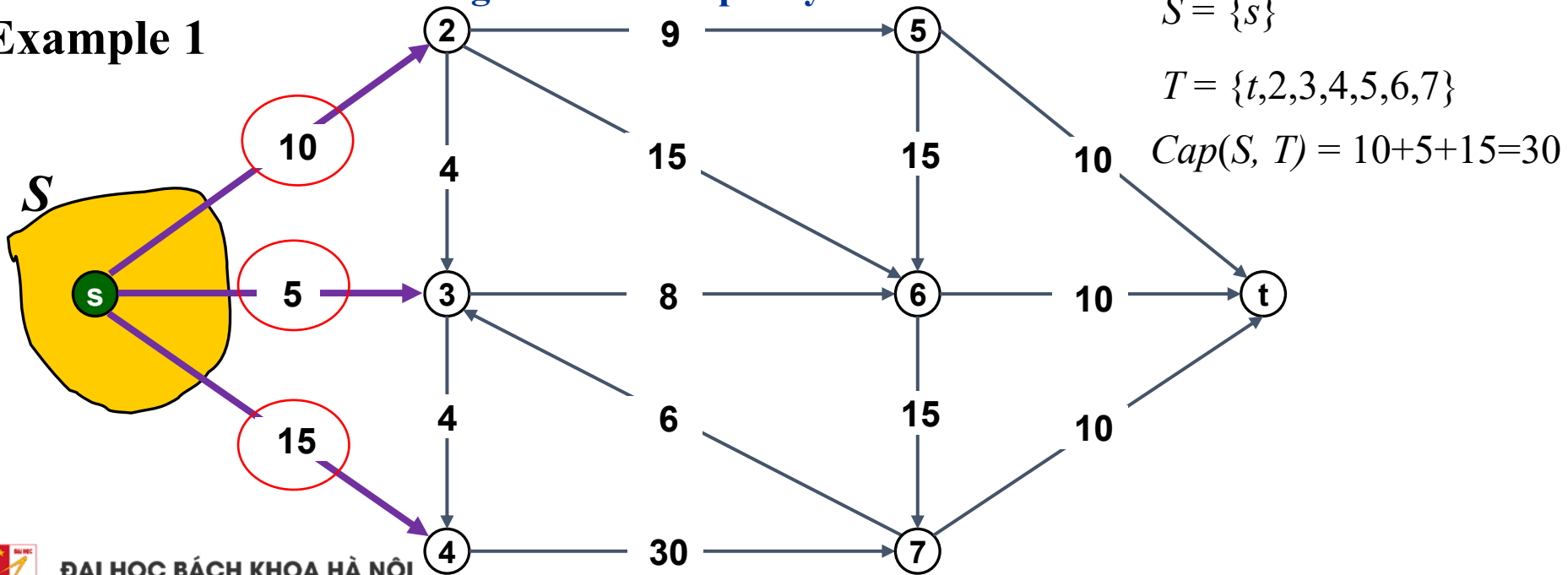
- Capacity $cap(S, T)$ of the cut (S, T) is:

$$cap(S, T) = \sum_{e \in S \rightarrow T} c(e),$$

where $S \rightarrow T := \{(v, w) \in E : v \in S, w \in T\}$

Min cut is the cut having minimum capacity.

Example 1



Flow through the cut

Definition. Assume f is a flow in network and (S, T) is a cut. We call **value of flow through the cut (S, T)** is the value

$$\sum_{e \in S \rightarrow T} f(e) - \sum_{e \in T \rightarrow S} f(e) \quad 6+0+8+10 \quad 0$$

where: $S \rightarrow T = \{(v, w) \in E: v \in S, w \in T\}$

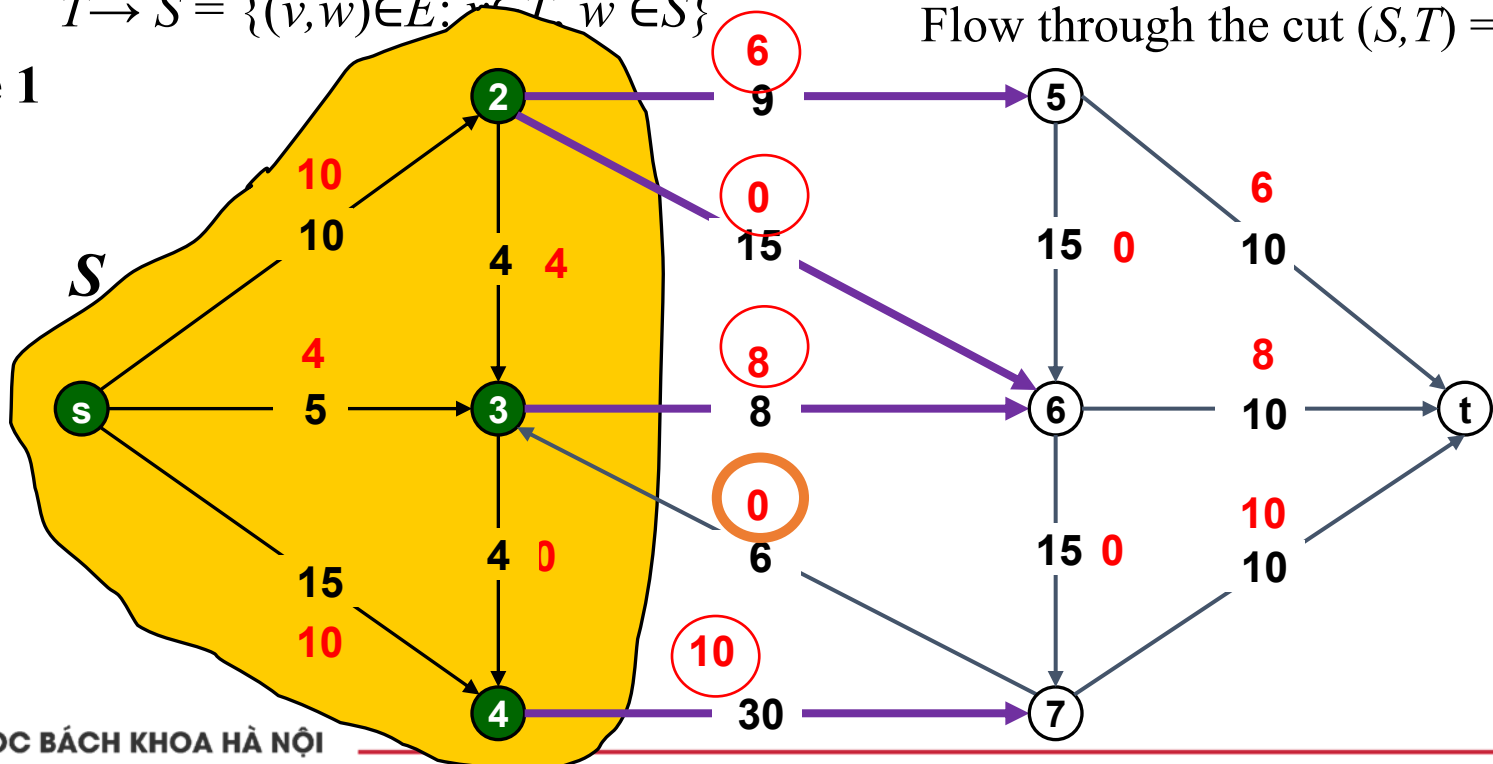
$T \rightarrow S = \{(v, w) \in E: v \in T, w \in S\}$

$S = \{s, 2, 3, 4\}$

$T = \{5, 6, 7, t\}$

Flow through the cut $(S, T) = ???$

Example 1



Flow and cut

Lemma 1. Assume f is flow, and (S, T) is a cut. Then

Flow through this cut is equal to the value of flow f :

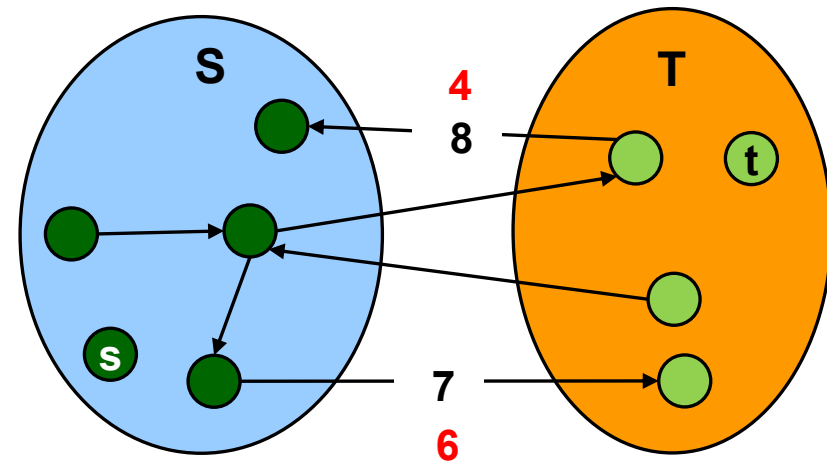
$$\sum_{e \in S \rightarrow T} f(e) - \sum_{e \in T \rightarrow S} f(e) = \sum_{e \in E^+(s)} f(e) = \sum_{e \in E^-(t)} f(e) = \text{val}(f)$$

Flow and cut

Lemma 2. Assume f is a flow, (S, T) is a cut. Then
$$\text{val}(f) \leq \text{cap}(S, T).$$

Proof

$$\begin{aligned} \text{val}(f) &= \sum_{e \in S \rightarrow T} f(e) - \sum_{e \in T \rightarrow S} f(e) \\ &\leq \sum_{e \in S \rightarrow T} f(e) \\ &\leq \sum_{e \in S \rightarrow T} c(e) \\ &= \text{cap}(S, T) \end{aligned}$$



Max Flow and Min Cut

Corollary. Assume f is a flow, (S, T) is a cut. If $\text{val}(f) = \text{cap}(S, T)$ then f is maximum flow and (S, T) is minimum cut.

Proof: Consider f' is arbitrary flow and (S', T') is arbitrary cut.
Using Lemma 2, we have:

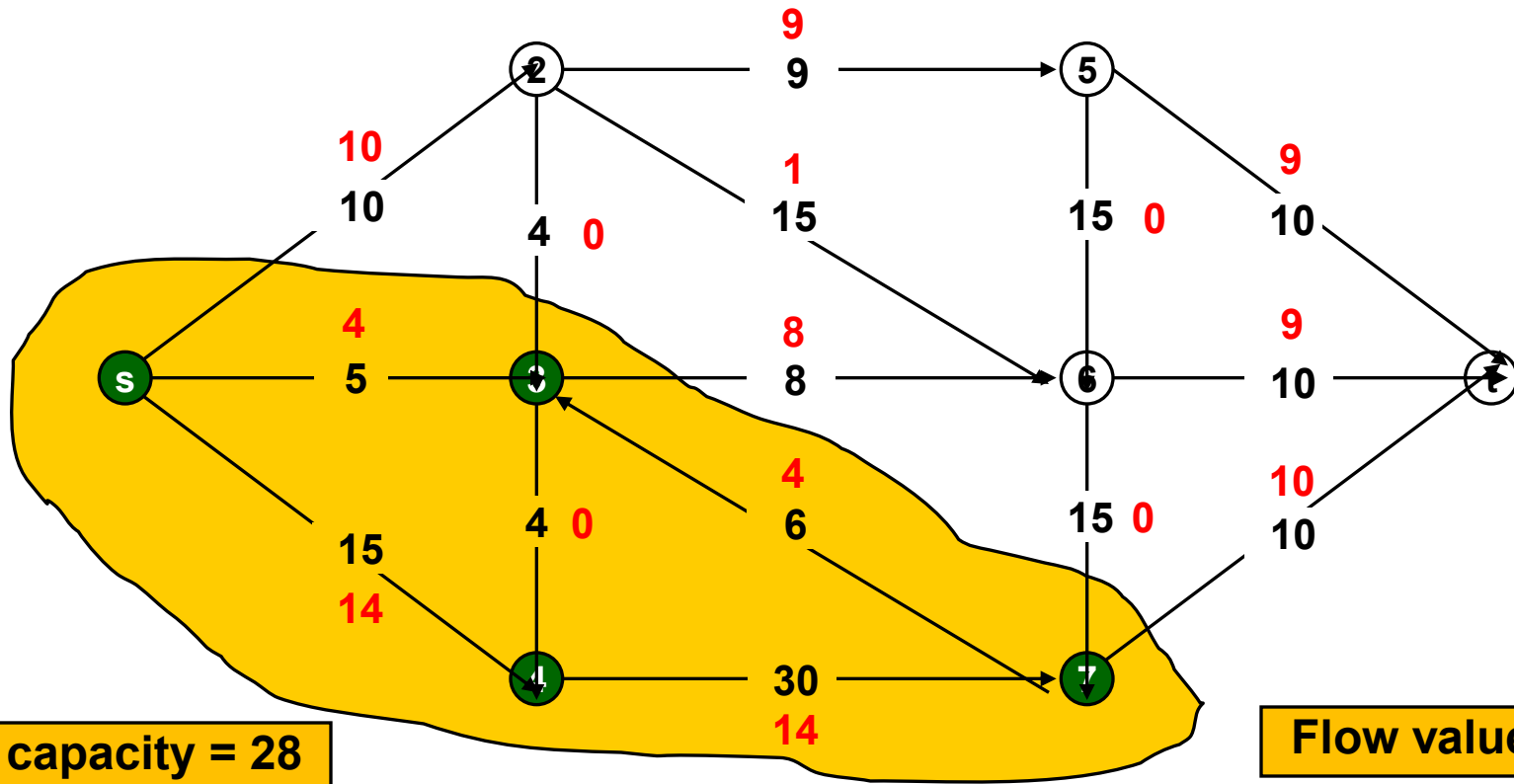
$$\text{val}(f') \leq \text{cap}(S, T) = \text{val}(f) \leq \text{cap}(S', T')$$

f' is arbitrary flow $\rightarrow$ f is max flow
Lemma 2. Assume f is a flow, and (S, T) is a cut. Then
 $\text{val}(f') \leq \text{val}(f)$ $\text{val}(f) \leq \text{cap}(S, T)$

(S', T') is arbitrary cut
 $\text{cap}(S, T) \leq \text{cap}(S', T') \rightarrow (S, T)$ is min cut

Max-Flow Min-Cut Theorem

Theorem (Ford-Fulkerson, 1956): In a network, value of max flow is equal to capacity of min cut.



Contents

1. Problem description and applications
2. Cut
- 3. Residual graph and Augmenting path**
4. Ford-Fulkerson algorithm
5. Edmond-Karp algorithm
6. Some applications



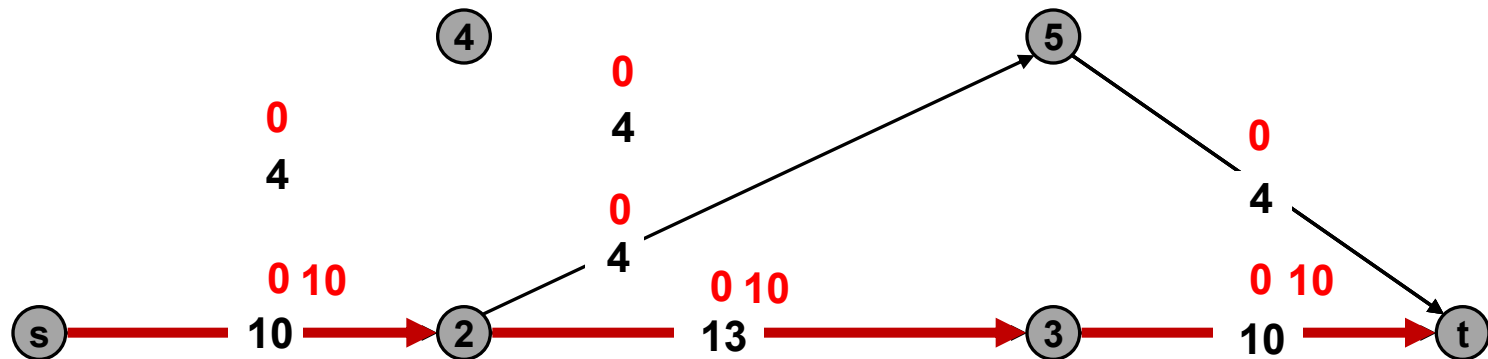
Greedy algorithm

Greedy algorithm:

- Starting with flow 0 (value of flow = 0).
- Find path P from s to t such that each edge e satisfies $f(e) < c(e)$.
- Augment flow along P .
- Repeat until can not find P .

$s \rightarrow 4 \rightarrow 3 \rightarrow t$???

$s \rightarrow 2 \rightarrow 5 \rightarrow t$???

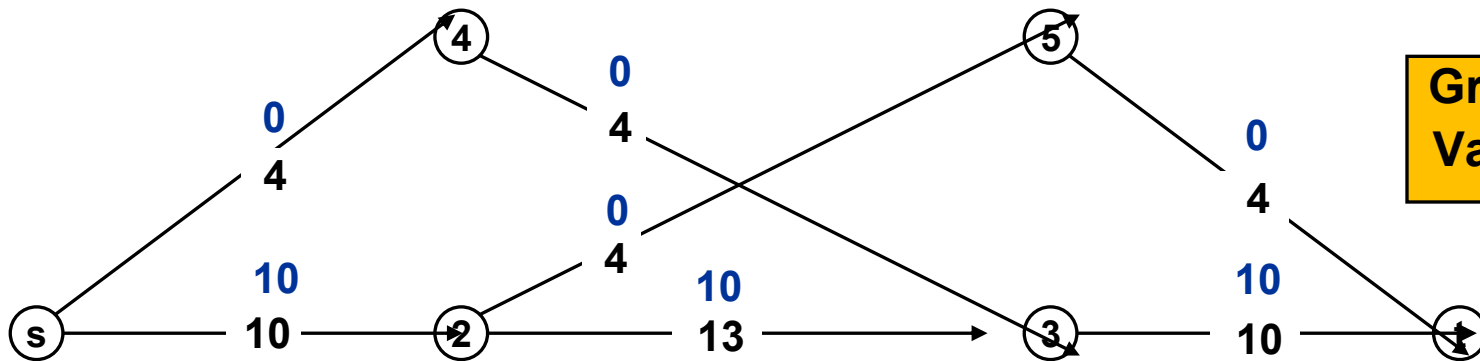


Value of flow = 10

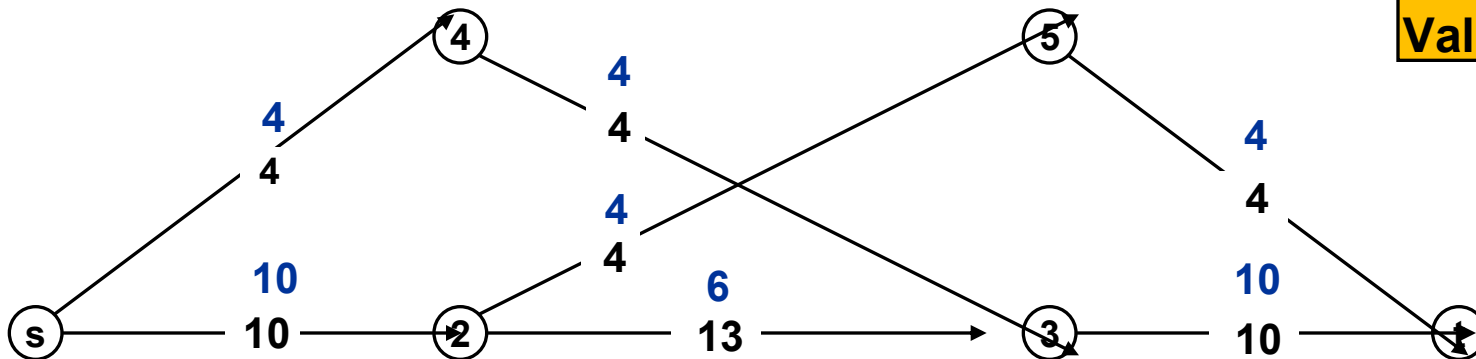
Greedy algorithm



Greedy algorithm does not provide optimal solution.



**Greedy algorithm:
Value of flow = 10**

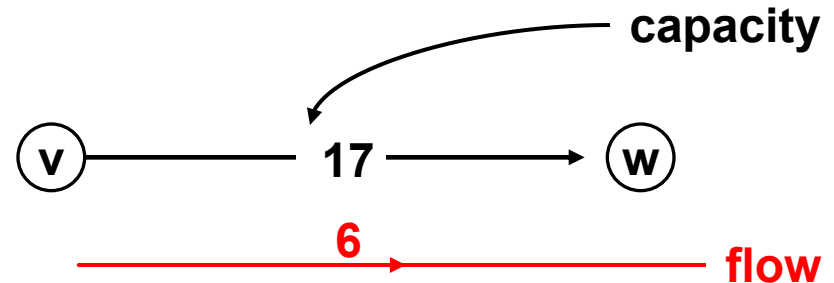


**Optimal:
Value of flow = 14**

Residual graph

Network $G = (V, E)$.

- Arc $e = (v, w) \in E$
- Flow $f(e)$
- Capacity $c(e)$

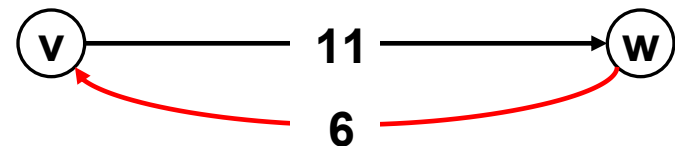


$$e = (u, v) \Rightarrow e^R = (v, u)$$

Residual graph: $G_f = (V, E_f)$.

- $E_f = \{e: f(e) < c(e)\} \cup \{e^R: f(e) > 0\}$
- Capacity of each arc e

$$c_f(e) = \begin{cases} c(e) - f(e) & \text{if } e \in E \\ f(e) & \text{if } e^R \in E \end{cases}$$

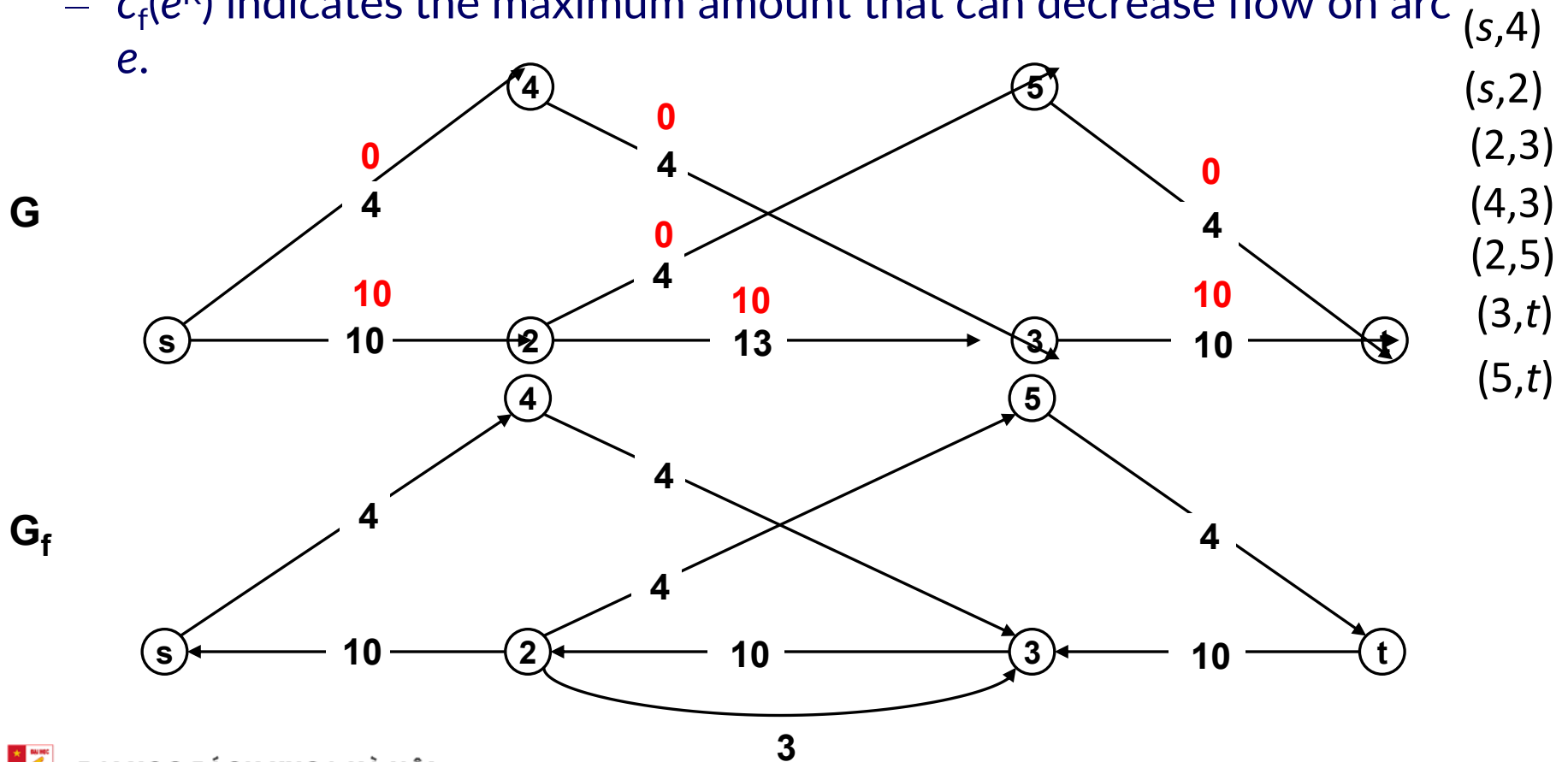


Residual graph - Example

Residual graph: $G_f = (V, E_f)$.

- $E_f = \{e : f(e) < c(e)\} \cup \{e^R : f(e) > 0\}$.
- $c_f(e)$ indicates the maximum amount that can increase flow on arc e .
- $c_f(e^R)$ indicates the maximum amount that can decrease flow on arc e .

$$c_f(e) = \begin{cases} c(e) - f(e) & \text{if } e \in E \\ f(e) & \text{if } e^R \in E \end{cases}$$



Residual graph - Example

Residual graph: $G_f = (V, E_f)$.

- $E_f = \{e : f(e) < c(e)\} \cup \{e^R : f(e) > 0\}$.
- $c_f(e)$ indicates the maximum amount that can increase flow on arc e .
- $c_f(e^R)$ indicates the maximum amount that can decrease flow on arc e .

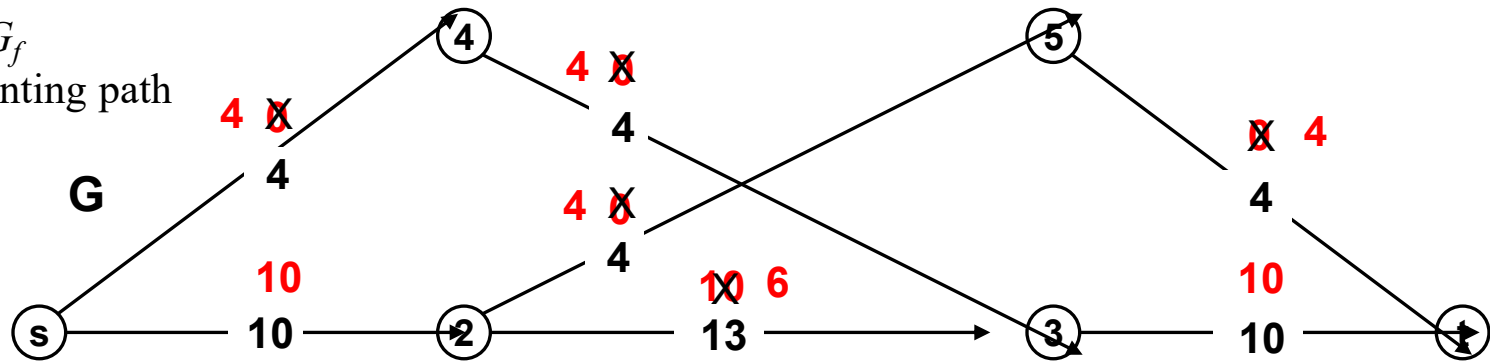
Augmenting path = path from s to t on the residual graph G_f .

Capacity of the path: $c_f(P) = \min \{c_f(e) : e \in P\}$.

Build new G_f

Find augmenting path

On new G_f

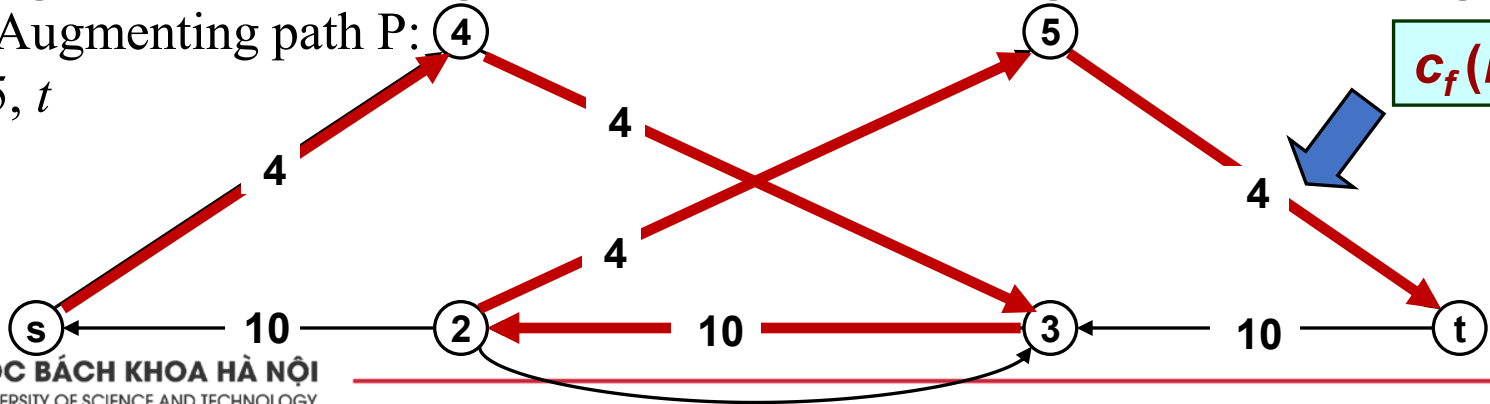


(s,4)
(s,2)
(2,3)
(4,3)
(2,5)
(3,t)
(5,t)

Example: Augmenting path P:

$s, 4, 3, 2, 5, t$

G_f



$c_f(P) = 4$

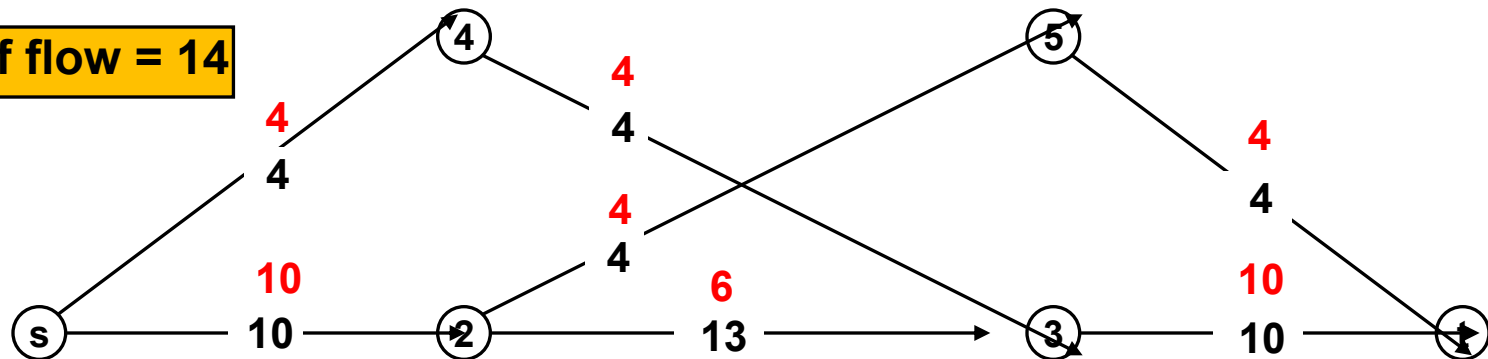
Augmenting path

Augmenting path = path from s to t on the residual graph.

- Max flow $\Leftrightarrow$ could not find any augmenting path???

Value of flow = 14

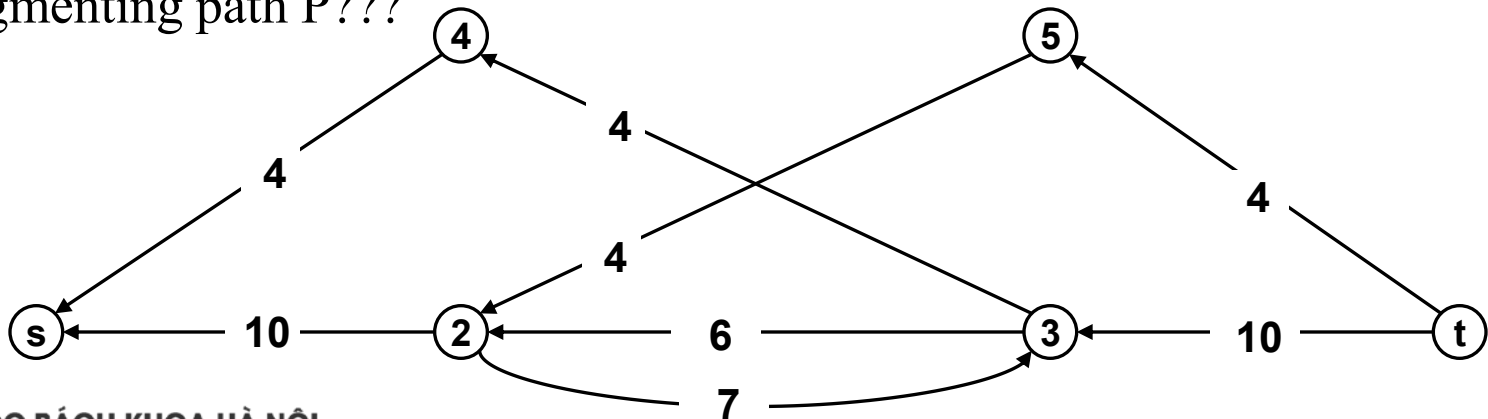
G



(s,4)
(s,2)
(2,3)
(4,3)
(2,5)
(3,t)
(5,t)

Find augmenting path P???

G_f



Theorem about maximum flow and minimum cut

Augmenting path Theorem (Ford-Fulkerson, 1956): Flow is maximum if and only if there does not exist augmenting path on network.

Max flow and min cut Theorem (Ford-Fulkerson, 1956): the maximum possible flow in a network (from source to sink) is exactly equal to the minimum capacity of all possible cuts.

We will prove the following Theorem:

Theorem. Assume f is a flow in network. The following three statements are equivalent

- (i) Can find the cut (S, T) such that $\text{val}(f) = \text{cap}(S, T)$.
- (ii) f is maximum flow.
- (iii) Could not find augmenting path to augment value flow f .

Contents

1. Problem description and applications
2. Cut
3. Residual graph and Augmenting path
- 4. Ford-Fulkerson algorithm**
5. Edmond-Karp algorithm
6. Some applications



Ford – Fulkerson algorithm

Augmenting flow f along path P

```
float Augment( $f, P$ )
{
     $b \leftarrow c_f(P)$ 
    FOR  $e \in P$ 
        IF ( $e \in E$ )
             $f(e) \leftarrow f(e) + b$ 
        ELSE
             $f(e^R) \leftarrow f(e) - b$ 
    RETURN  $f$ 
}
```

Ford-Fulkerson algorithm

```
float Ford_Fulkerson( $G, c, s, t$ )
{
    FOR  $e \in E$  // initialize flow  $f$  to 0
         $f(e) \leftarrow 0$ 
     $G_f \leftarrow$  residual graph for  $f$ 

    WHILE (there exists an augmenting path  $P$ )
    {
         $f \leftarrow$  augment( $f, P$ )
        Update residual graph  $G_f$ 
    }
    RETURN  $f$ 
}
```



Computation time

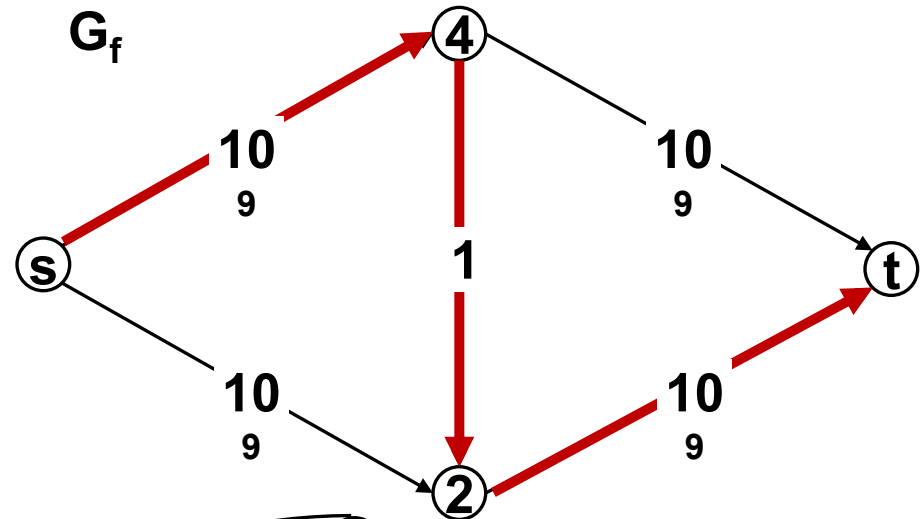
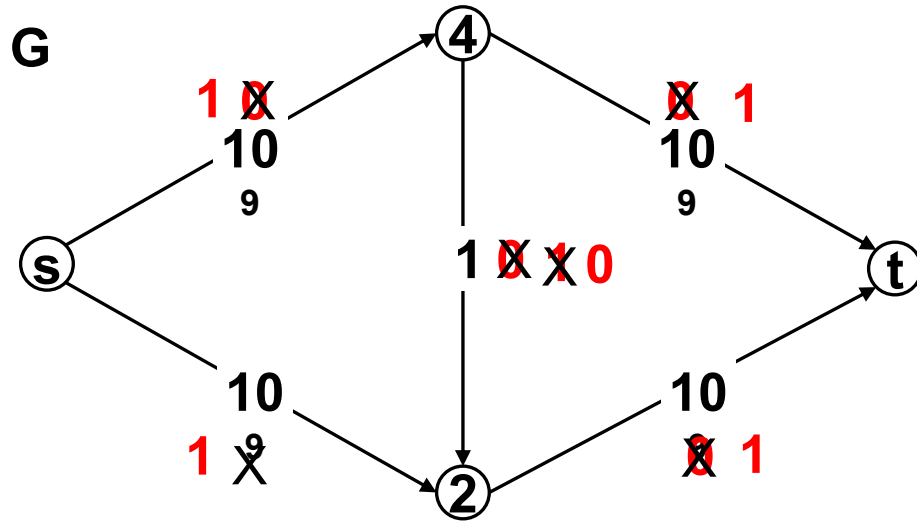
Question: Is the Ford-Fulkerson algorithm a polynomial algorithm? (algorithm with computation time is bounded by a fixed degree polynomial of the input length)

- **Answer:** Not at all. If the maximum capacity is C then the algorithm may have to do C iterations.

The following example shows how the algorithm can take a lot of iteration

Ford – Fulkerson algorithm is not polynomial algorithm

Augmenting path $P: s, 4, 2, t$



Algorithm executes alternating augmenting flow along two paths P and P'

$(s, 4)$

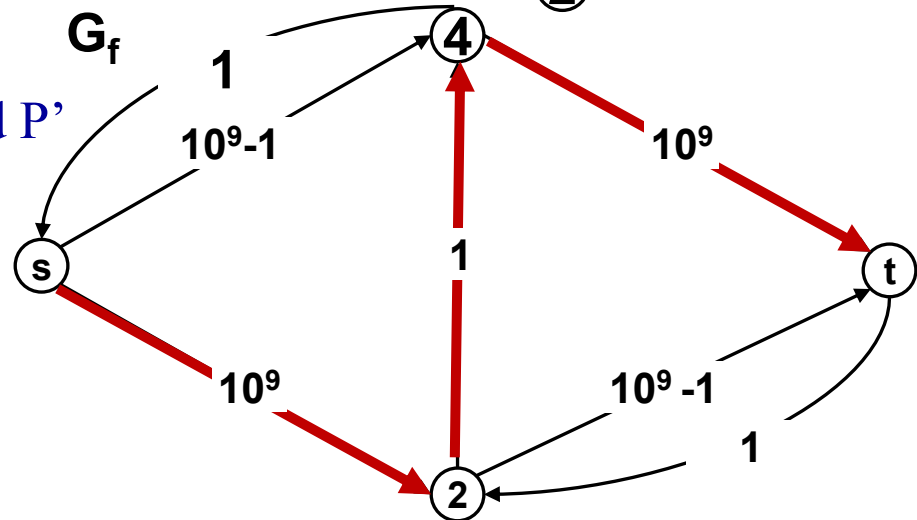
$(s, 2)$

$(4, 2)$

$(2, t)$

$(4, t)$

2×10^9 iterations



Augmenting path $P': s, 2, 4, t$

Computation time

Question: Is the Ford-Fulkerson algorithm a polynomial algorithm? (algorithm with computation time is bounded by a fixed degree polynomial of the input length)

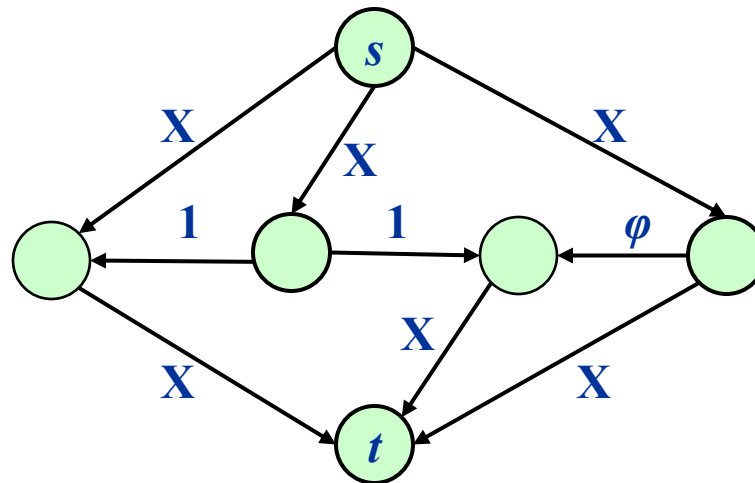
- **Answer:** Not at all. If the maximum capacity is C then the algorithm may have to do C iterations.

If the capacities of arcs on network is real, there is an example that the Ford-Fulkerson algorithm does not stop.

Zwick constructs examples showing that the algorithm may not stop, if the capacity of arcs is irrational

Example: Ford – Fulkerson algorithm does not terminate

Zwick constructs examples showing that the algorithm may not stop, if the capacity of arcs is irrational



There are 6 arcs with capacity = X , 2 arcs with capacity = 1 and one arc with capacity = $\phi = (\sqrt{5}-1)/2 \approx 0.618034\dots$

Ford – Fulkerson algorithm

Assumption: All capacities on arcs are integers in the range $[0, C]$.

Immutability: Each value of flow value $f(e)$ and capacity $c_f(e)$ are always integer during algorithm execution.

Theorem: F-F algorithm stops after not more than $\text{val}(f^*) \leq nC$ iterations.

Proof.

After each augmenting flow, value of flow is augmented by at least 1

Corollary. Computation time of Ford-Fulkerson algorithm is $O(m.n.C)$

Corollary. If $C = 1$, then the computation time of algorithm is $O(mn)$.



Contents

1. Problem description and applications
2. Cut
3. Residual graph and Augmenting path
4. Ford-Fulkerson algorithm
5. **Edmond-Karp algorithm**
6. Some applications



How to select the augmenting path?

Be careful when select augmenting path, because

- Several options may lead to an exponential algorithm.
- Smart choice leads to polynomial algorithm.
- If capacity is irrational number, the algorithm may not stop

The goal: select augmenting path such that:

- Can find the augmenting path effectively.
- The algorithm requires as few iterations as possible..

Select augmenting path with

- Maximum capacity. (fat path)
- Capacity is large enough. (capacity scaling)
- The number of edges along the way is the least. (shortest path)



Edmond-Karp algorithm

Applying BFS



5. Edmonds – Karp Algorithm

Edmonds and Karp, *JACM* 1972

- If the augmenting path is the shortest path from s to t , then the computation time of the algorithm is $O(|E|^2 |V|)$.



Ford-Fulkerson algorithm

```
float Ford_Fulkerson(G,c,s,t)
{
    FOR e ∈ E    // initialize flow to 0
        f(e) ← 0
    Gf ← residual graph for flow f

    WHILE (there exists an augmenting path P)
    {   f ← augment(f, P)
        update Gf
    }
    RETURN f
}
```



$$O(|E|^2 |V|)$$



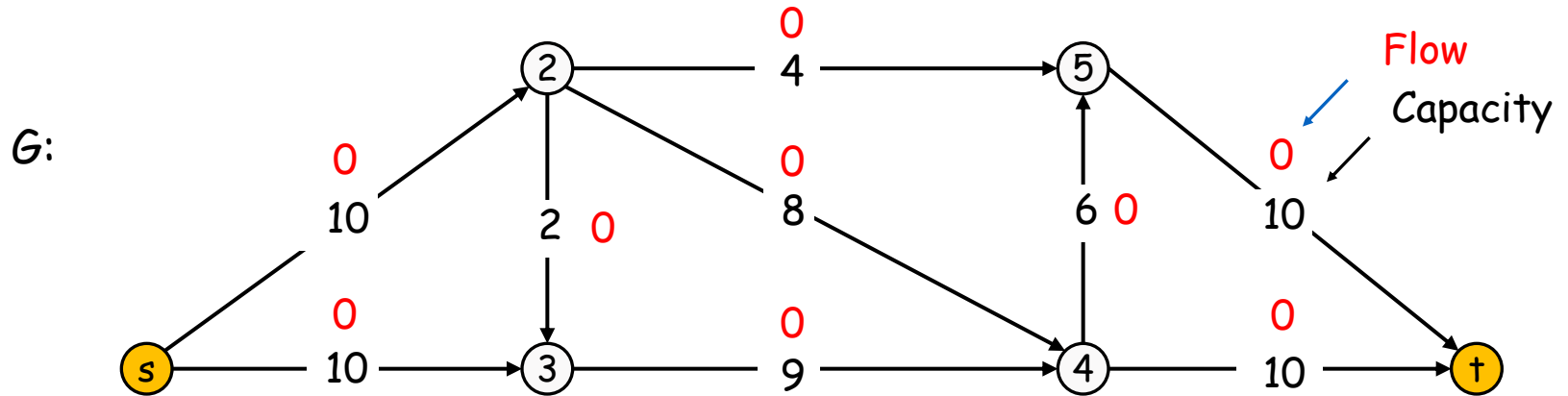
Edmonds – Karp algorithm

```
FOR e ∈ E
    f(e) ← 0
Gf ← residual graph for flow f

WHILE (there exists an augmenting path)
{
    find augmenting P by BFS
    f ← augment(f, P)
    update Gf
}
RETURN f
```



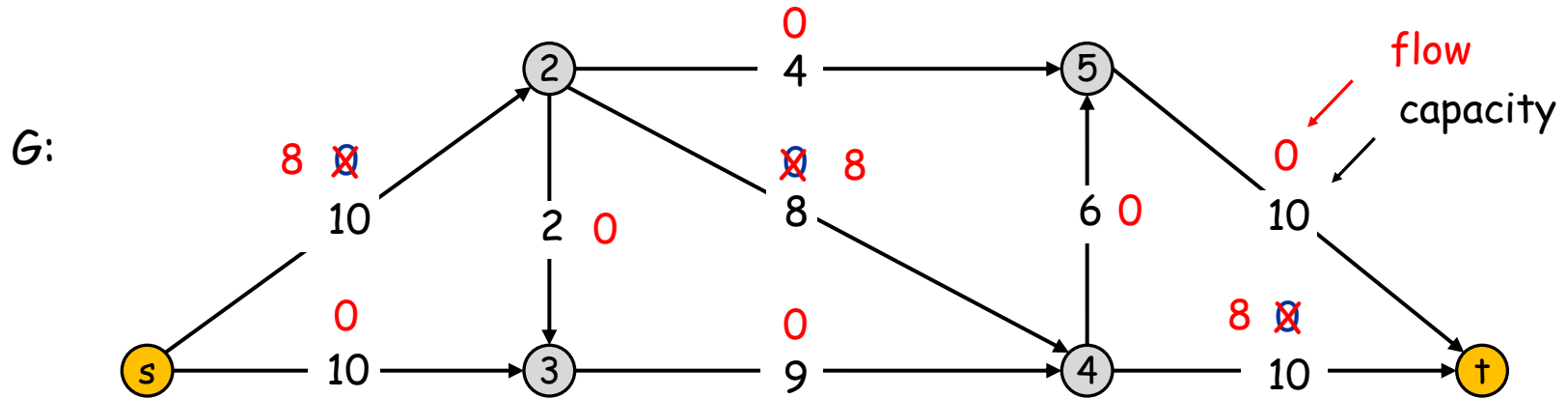
Edmonds-Karp Algorithm



Value of flow = 0

Build the residual graph G_f

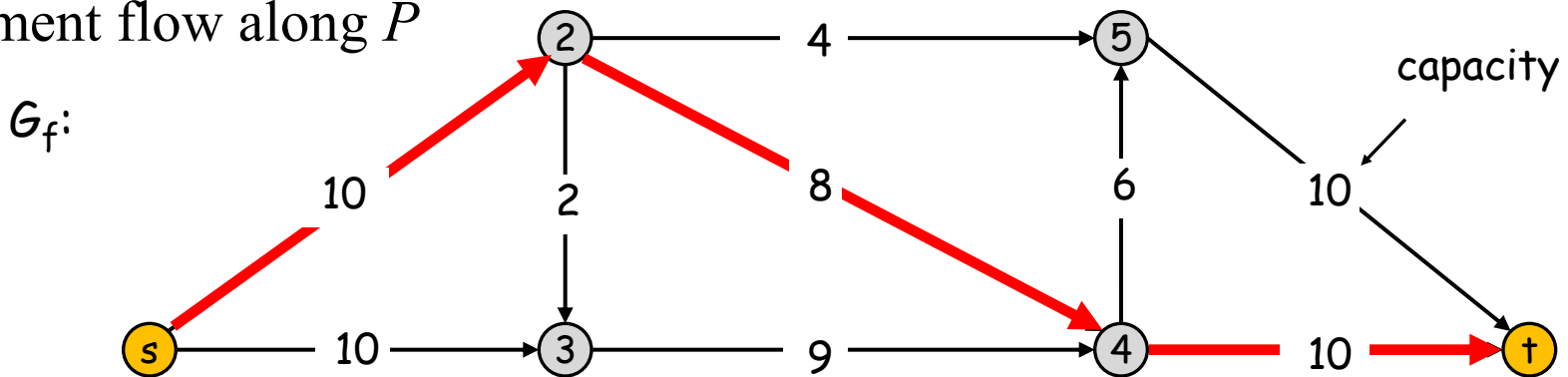
Edmonds-Karp Algorithm



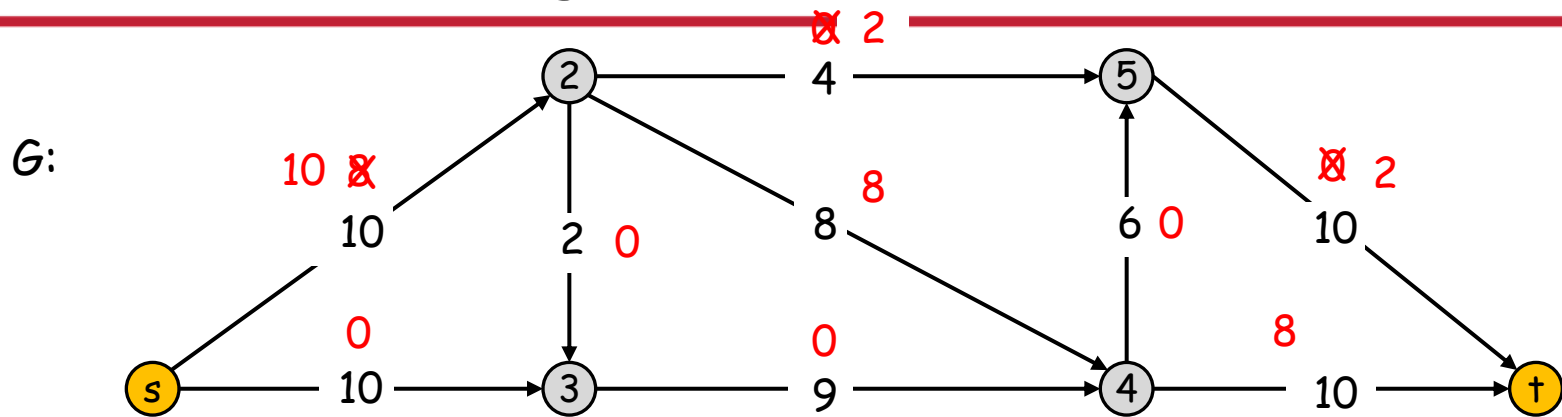
Value of flow = 8

Augmenting path P : $s, 2, 4, t$

Augment flow along P



Edmonds-Karp Algorithm



Build residual graph G_f

Augmenting path P : $s, 2, 5, t$ Augment flow along P

Value of flow = 10

$(s, 2)$

$(s, 3)$

$(2, 3)$

$(2, 5)$

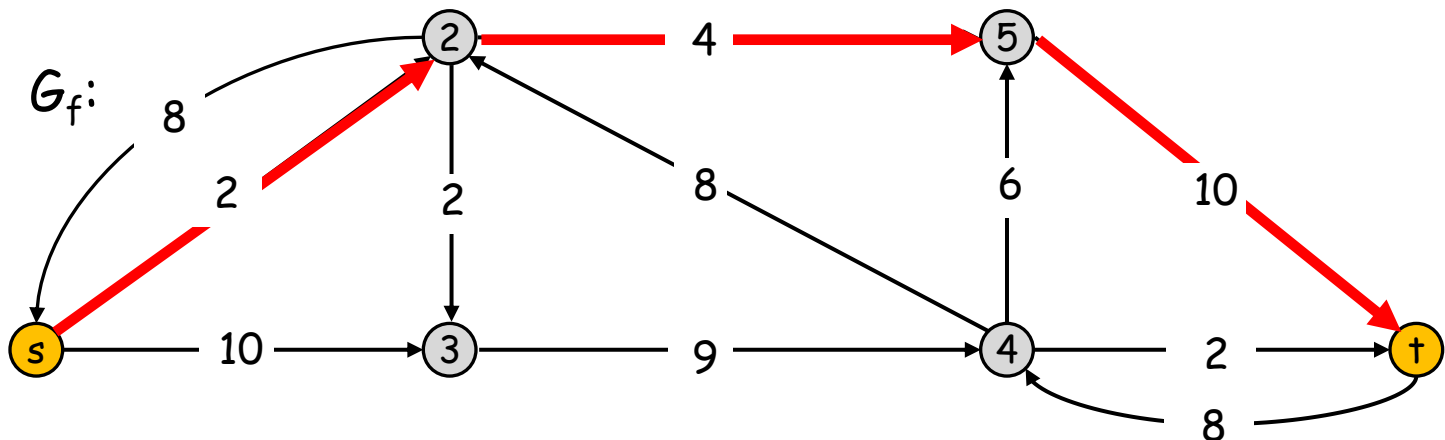
$(2, 4)$

$(3, 4)$

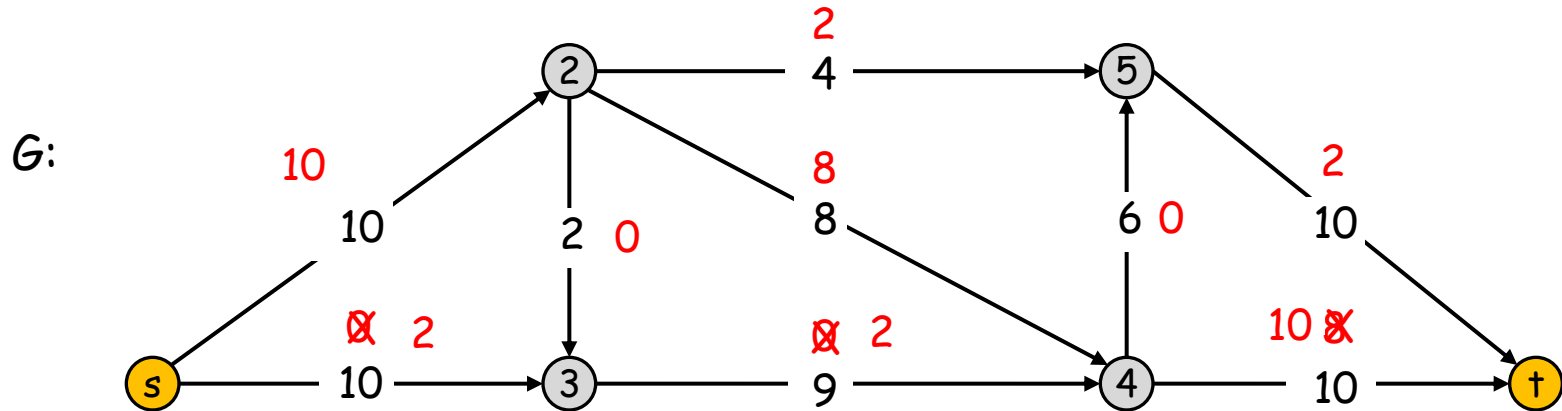
$(5, t)$

$(4, 5)$

$(4, t)$



Edmonds-Karp Algorithm

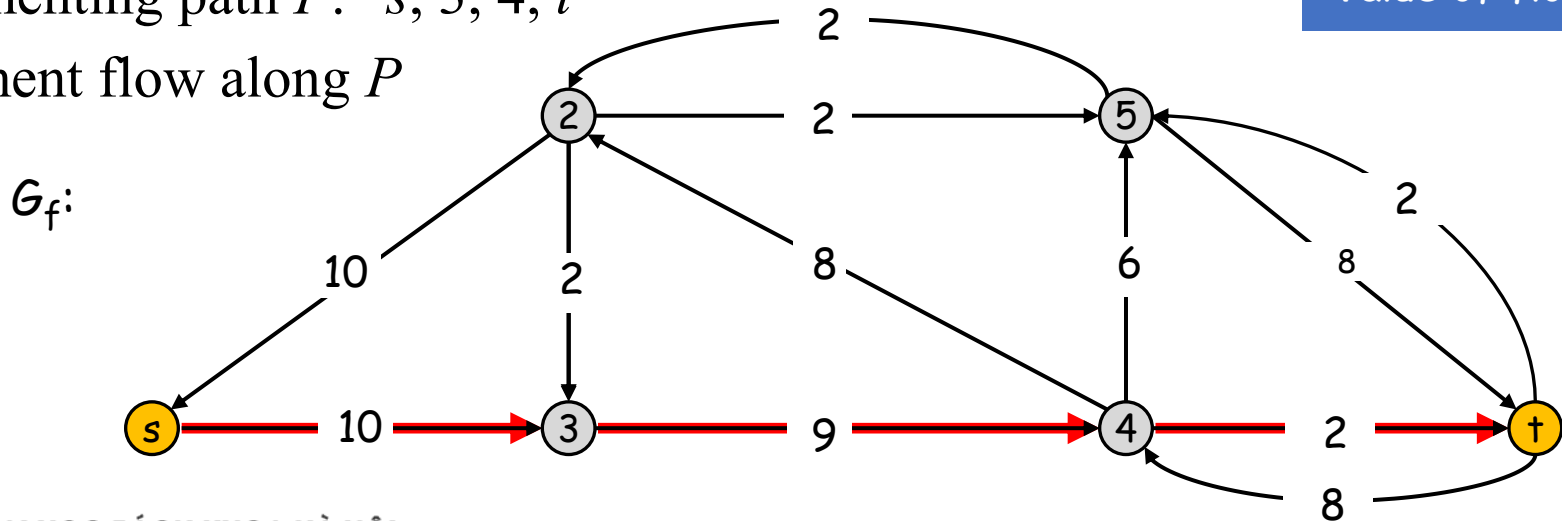


Build residual graph G_f

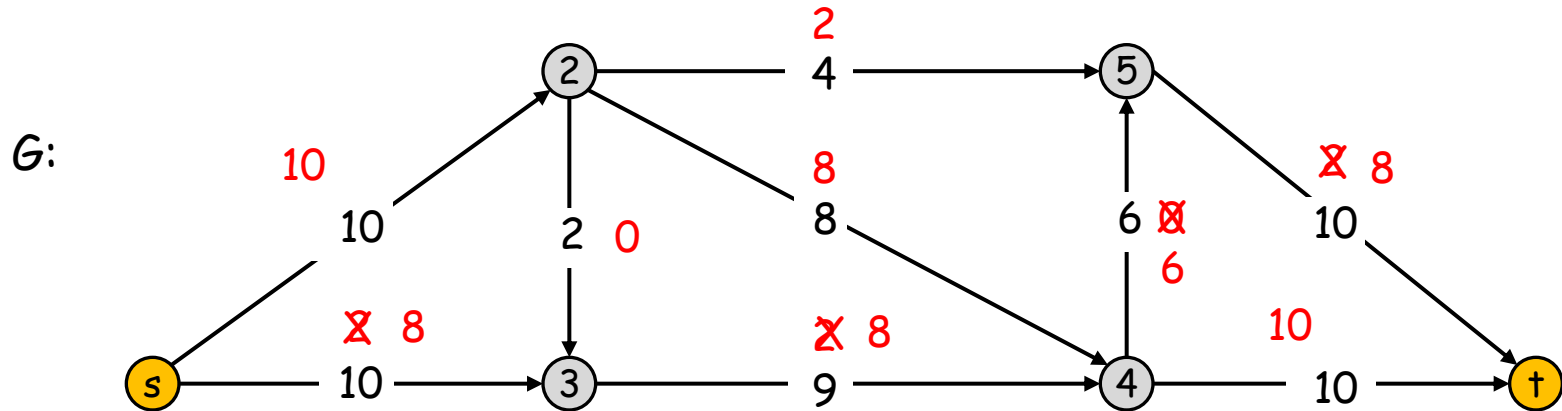
Augmenting path P : $s, 3, 4, t$

Augment flow along P

Value of flow = 12



Edmonds-Karp Algorithm

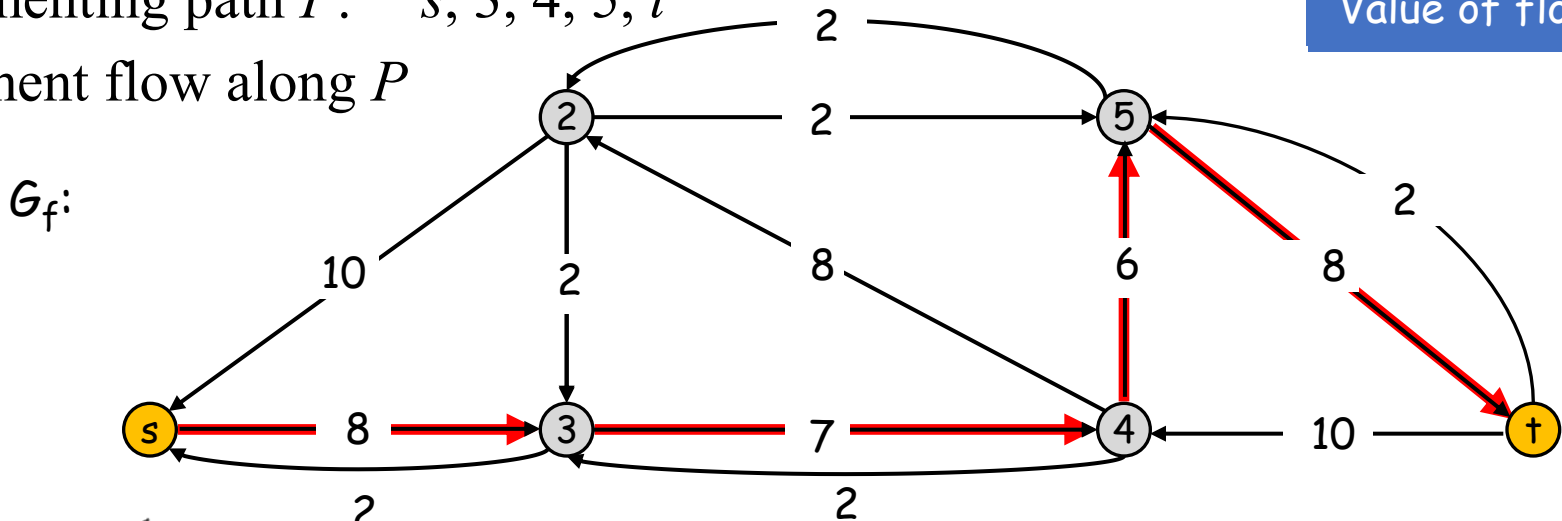


Build residual graph G_f

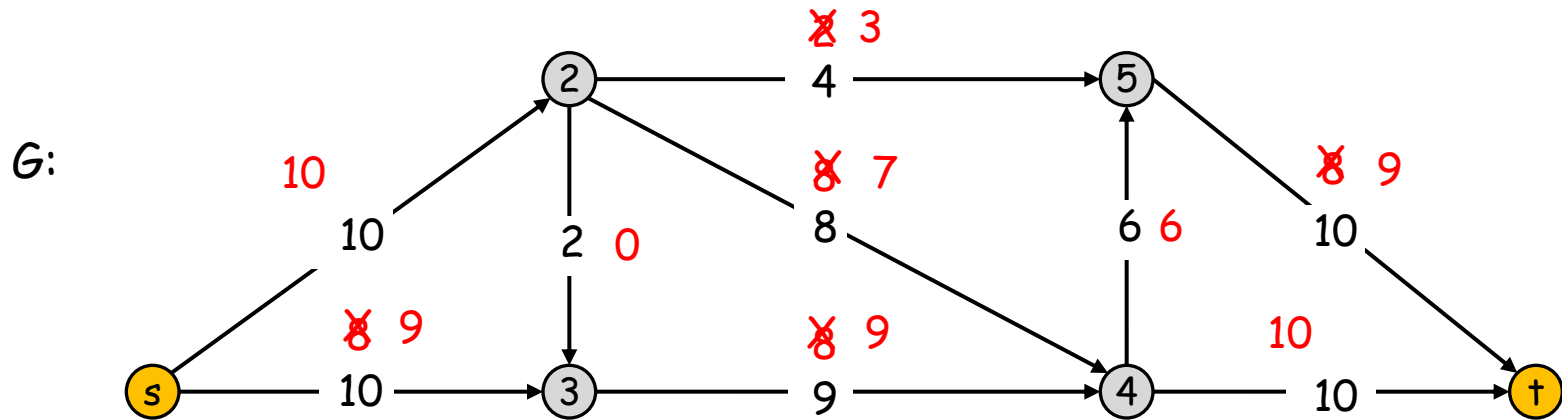
Augmenting path P : $s, 3, 4, 5, t$

Augment flow along P

Value of flow = 18



Edmonds-Karp Algorithm

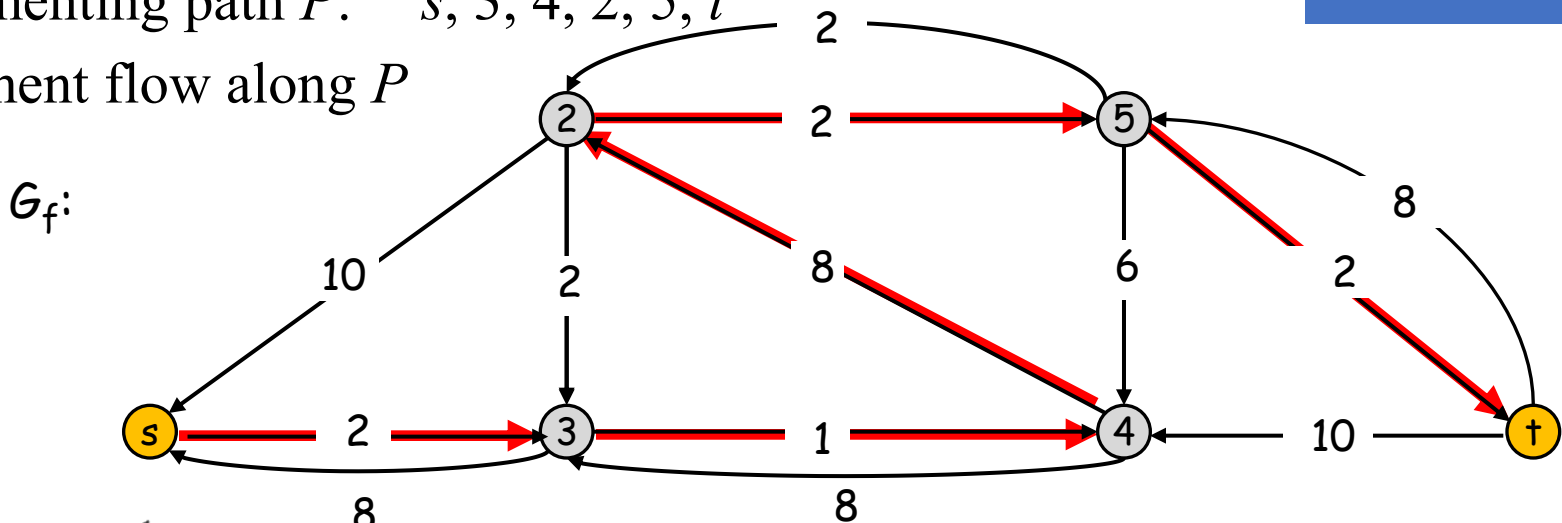


Build residual graph G_f

Augmenting path P : $s, 3, 4, 2, 5, t$

Augment flow along P

Value of flow = 19



Conclusion:

In graph G_f , S = set of vertices reached from s by BFS(s)

- Min cut (S , T):

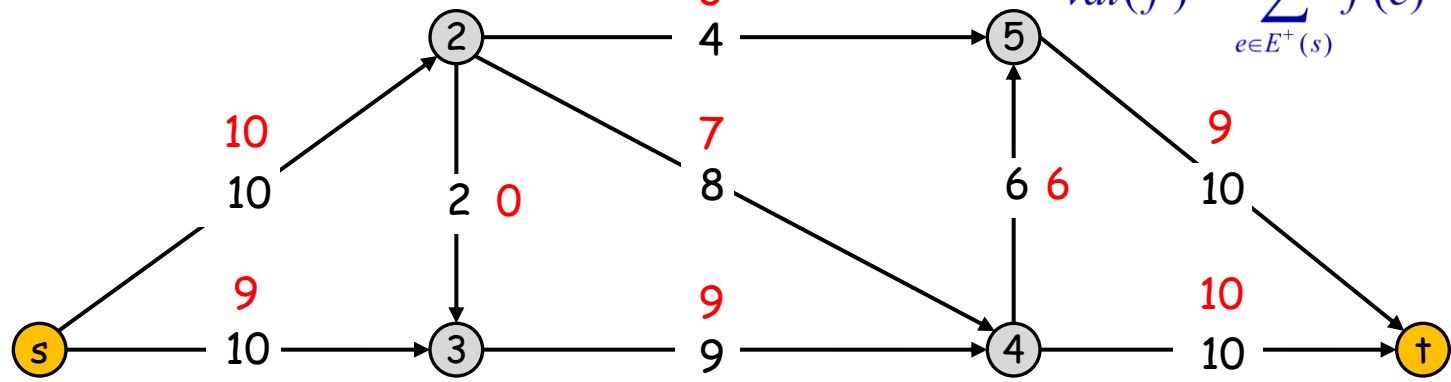
$S = \{s, 3\}; T = \{2,4,5,t\}$

- Max flow:

$val(f) = 10 + 9 = 19$

$val(f) = \sum_{e \in E^+(s)} f(e) = \sum_{e \in E^-(t)} f(e)$

G :



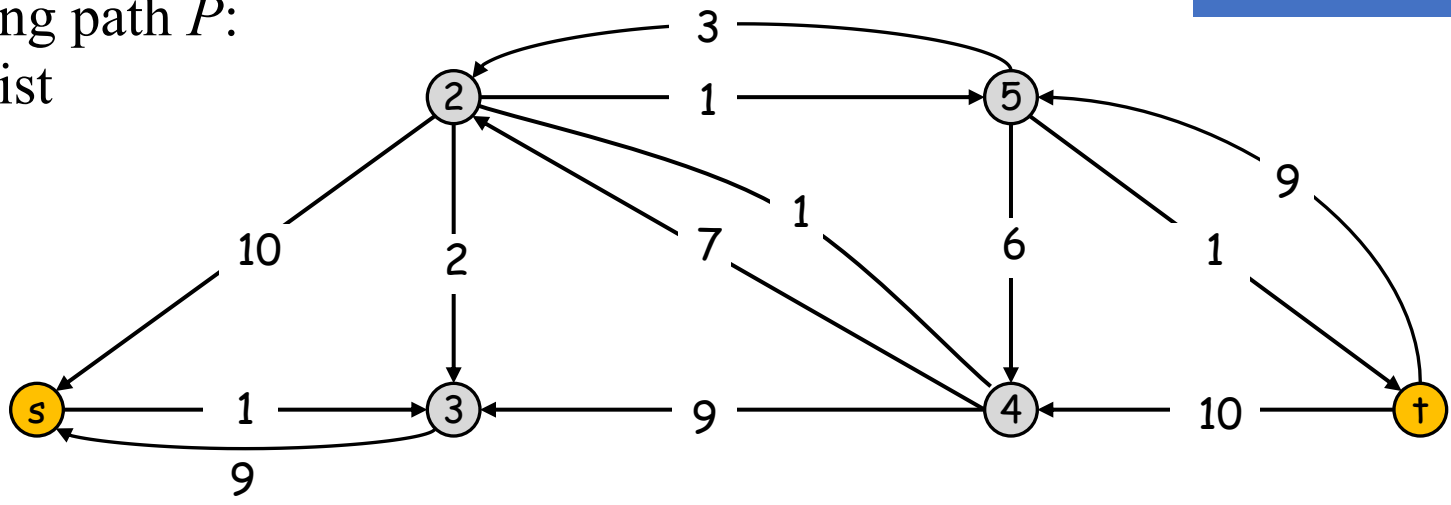
Build residual graph G_f

Augmenting path P :

Not exist

Value of flow = 19

G_f :



Applications

- Some general flow problems:
 - The problem with multiple sources and sinks
 - The problem with limited capacities at nodes
- Airline scheduling
- Image segmentation
- Bipartite matching
- Data mining
- Project selection
- Network connectivity
- ...



A decorative graphic on the left side of the slide. It features a dark blue background with a large, stylized circular pattern composed of many small red dots. The dots are arranged in concentric, slightly irregular rings, creating a sense of depth and movement. The word "HUST" is centered within this pattern.

HUST

THANK YOU !